Rapport de TP 3 — Sécurité des conteneurs et monitoring en runtime

Université : INSAT
Classe : GL5
Année universitaire : 2026/2027

Équipe :

- Ahmed Jammoussi
- Mohamed Taher Ben Hassine
- Yassine Sallemi
- Youssef Fadhloun

---

1. Objectif

Le TP 3 porte sur trois contrôles qui interviennent après le code : l'image Docker, les secrets au déploiement, et la détection d'un comportement anormal une fois le conteneur démarré.

L'application est la Notes API Flask déjà durcie aux TP 1 et 2. Elle lit une clé `API_KEY` dans l'environnement et expose `/status`, qui indique seulement si la clé est présente et quelle est sa longueur. La valeur n'est jamais renvoyée.

Le travail se fait en trois temps :

1. Scanner une image volontairement faible avec Trivy, puis reconstruire une image minimale, non-root, sans secret.
2. Déployer l'image durcie sur Minikube et injecter `API_KEY` par un Secret Kubernetes.
3. Installer Falco sur le cluster, ouvrir un shell dans le pod, et montrer l'alerte.

---

2. Image faible

Le premier Dockerfile part de `python:3.9`, n'ajoute pas d'utilisateur, et écrit le secret dans l'image avec `ENV API_KEY=super-secret-lab-key`.

TP3 › Dockerfile.vulnerable › ...

```dockerfile
# Intentionally weak image, used only as the "before" scan baseline.
# Problems: old fat base, root user, secret baked into the image layers.
FROM python:3.9

ENV API_KEY=super-secret-lab-key
ENV APP_HOST=0.0.0.0

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .

EXPOSE 5000
CMD ["python", "app.py"]

```

Trois problèmes en découlent :

- la base est une Debian complète et un Python ancien, donc une grande surface de paquets ;
- le processus tourne en root ;
- la clé reste lisible par `docker inspect` pour quiconque obtient l'image.

La construction de `notes-api:vulnerable` le montre directement. `docker inspect` laisse le champ utilisateur vide (le processus est root) et affiche `API_KEY=super-secret-lab-key` dans l'environnement de l'image. Docker émet aussi l'avertissement `SecretsUsedInArgOrEnv` : une instruction `ENV` ne doit pas porter de donnée sensible.

```
(base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/TP3$ docker build -f Dockerfile.vulnerable -t notes-api:vulnerable .
docker inspect notes-api:vulnerable --format '{{.Config.User}} {{.Config.Env}}'
[+] Building 1.4s (10/10) FINISHED                                                                                 docker:default
 => [internal] load build definition from Dockerfile.vulnerable                                                             0.0s
 => => transferring dockerfile: 463B                                                                                        0.0s
 => [internal] load metadata for docker.io/library/python:3.9                                                               1.1s
 => [internal] load .dockerignore                                                                                           0.0s
 => => transferring context: 121B                                                                                           0.0s
 => [1/5] FROM docker.io/library/python:3.9@sha256:da5aee29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c             0.0s
 => => resolve docker.io/library/python:3.9@sha256:da5aee29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c             0.0s
 => [internal] load build context                                                                                           0.0s
 => => transferring context: 181B                                                                                           0.0s
 => CACHED [2/5] WORKDIR /app                                                                                               0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                    0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                         0.0s
 => CACHED [5/5] COPY app.py .                                                                                              0.0s
 => exporting to image                                                                                                      0.1s
 => => exporting layers                                                                                                     0.0s
 => => exporting manifest sha256:2f12fefbf224478db1bfb362b37615337a86a22bcc0e67ada665bf28a6dbddbb                           0.0s
 => => exporting config sha256:51f489f8779f27c9a566727ca0f4cfa4d6f17a9e9945ce5bb02368aefa6ea459                             0.0s
 => => exporting attestation manifest sha256:bf602464e5305b8cd8ce283daa4357b2e053c54a7904bcd318e5d74aa0b913a3               0.0s
 => => exporting manifest list sha256:e1998e1a19e5d7a7d755fce849f01999daa5510e636680baa0b5cf07d6875b4d                      0.0s
 => => naming to docker.io/library/notes-api:vulnerable                                                                     0.0s
 => => unpacking to docker.io/library/notes-api:vulnerable                                                                  0.0s

 1 warning found (use docker --debug to expand):
 - SecretsUsedInArgOrEnv: Do not use ARG or ENV instructions for sensitive data (ENV "API_KEY") (line 5)
 [PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin LANG=C.UTF-8 GPG_KEY=E3FF2839C048B25C084DEBE9B
26995E310250568 PYTHON_VERSION=3.9.25 PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834 API_KEY=supe
r-secret-lab-key APP_HOST=0.0.0.0]
```

Trivy, limité aux sévérités HIGH et CRITICAL, confirme la surface de la base. Sur `notes-api:vulnerable` (Debian 13.1), le système compte **2774** vulnérabilités, dont **2539 HIGH** et **235 CRITICAL**.

Report Summary

Target	Type	Vulnerabilities	Secrets
notes-api:vulnerable (debian 13.1)	debian	2774	-
usr/local/lib/python3.9/site-packages/MarkupSafe-3.0.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/blinker-1.9.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/click-8.1.8.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/flask-3.0.3.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/importlib_metadata-8.7.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/itsdangerous-2.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/jinja2-3.1.6.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.9/site-packages/pip-23.0.1.dist-info/METADATA	python-pkg	0	-

notes-api:vulnerable (debian 13.1)
==========
Total: 2774 (HIGH: 2539, CRITICAL: 235)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apt	CVE-2026-109518	HIGH	fixed	3.0.3	3.0.4	apt: apt: Remote code execution via hard link reuse during package download... https://avd.aquasec.com/nvd/cve-2026-109518
bsdutils	CVE-2026-53612			1:2.41-5	2.41.5-0+deb13u1	util-linux: util-linux: TOCTOU in the mount program when applying post-mount ownership/mode changes... https://avd.aquasec.com/nvd/cve-2026-53612
	CVE-2026-53613					util-linux: TOCTOU in the mount program via ancestor directory swap on target... https://avd.aquasec.com/nvd/cve-2026-53613
	CVE-2026-53614					util-linux: util-linux: SUID mount(8) allows nosuid/noexec bypass via LIBMOUNT_FORCE_MOUNT2 https://avd.aquasec.com/nvd/cve-2026-53614
	CVE-2026-53615					util-linux: util-linux: Partition metadata compromise via integer overflow in DOS partition parser...

Les paquets Python de l'application ajoutent **3 HIGH** et **0 CRITICAL** : `jaraco.context` (CVE-2026-23949) et `wheel` (CVE-2026-24049). Ce sont des paquets de métadonnées, pas le code Flask.

Python (python-pkg)
==========
Total: 3 (HIGH: 3, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
jaraco.context (METADATA)	CVE-2026-23949	HIGH	fixed	5.3.0	6.1.0	jaraco.context: jaraco.context: Path traversal via malicious tar archives https://avd.aquasec.com/nvd/cve-2026-23949
wheel (METADATA)	CVE-2026-24049			0.45.1	0.46.2	wheel: wheel: Privilege Escalation or Arbitrary Code Execution via malicious wheel file... https://avd.aquasec.com/nvd/cve-2026-24049

3. Image durcie

Le Dockerfile retenu change quatre points :

- base `python:3.12-slim-bookworm`, qui retire l'essentiel des paquets dont l'application n'a pas besoin ;
- utilisateur `appuser` (uid 10001), et `USER appuser` avant le `CMD` ;
- aucun `API_KEY` dans les instructions : la clé n'existe qu'au démarrage du conteneur ;
- `pip install --no-cache-dir` pour ne pas laisser le cache de téléchargement dans un layer.

```
...
# - slim base (fewer packages, smaller CVE surface)
# - no secrets in ENV or layers
# - non-root user
# - no pip cache
FROM python:3.12-slim-bookworm

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_HOST=0.0.0.0 \
    DB_PATH=/tmp/users.db

WORKDIR /app

RUN useradd --create-home --uid 10001 appuser

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt \
    && chown -R appuser:appuser /app

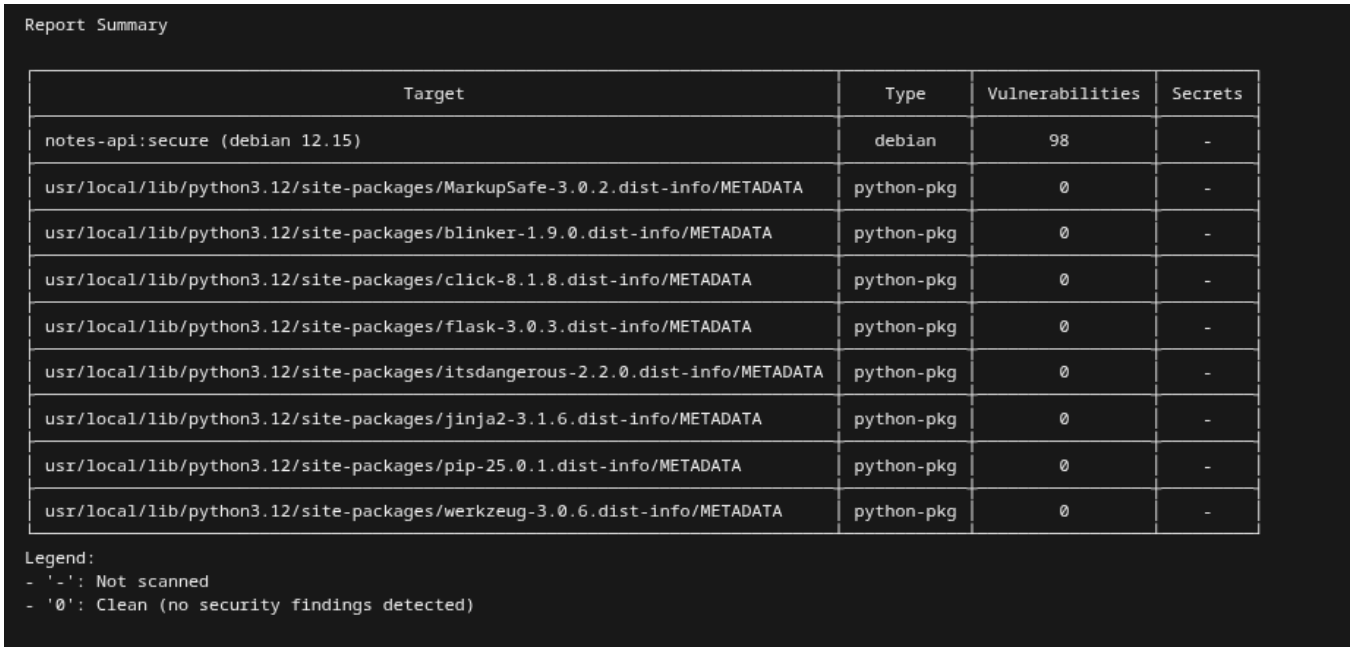
COPY --chown=appuser:appuser app.py .

USER appuser

EXPOSE 5000
CMD ["python", "app.py"]
```

Après construction, `docker inspect` montre `User=appuser` et une variable d'environnement sans `API_KEY`.

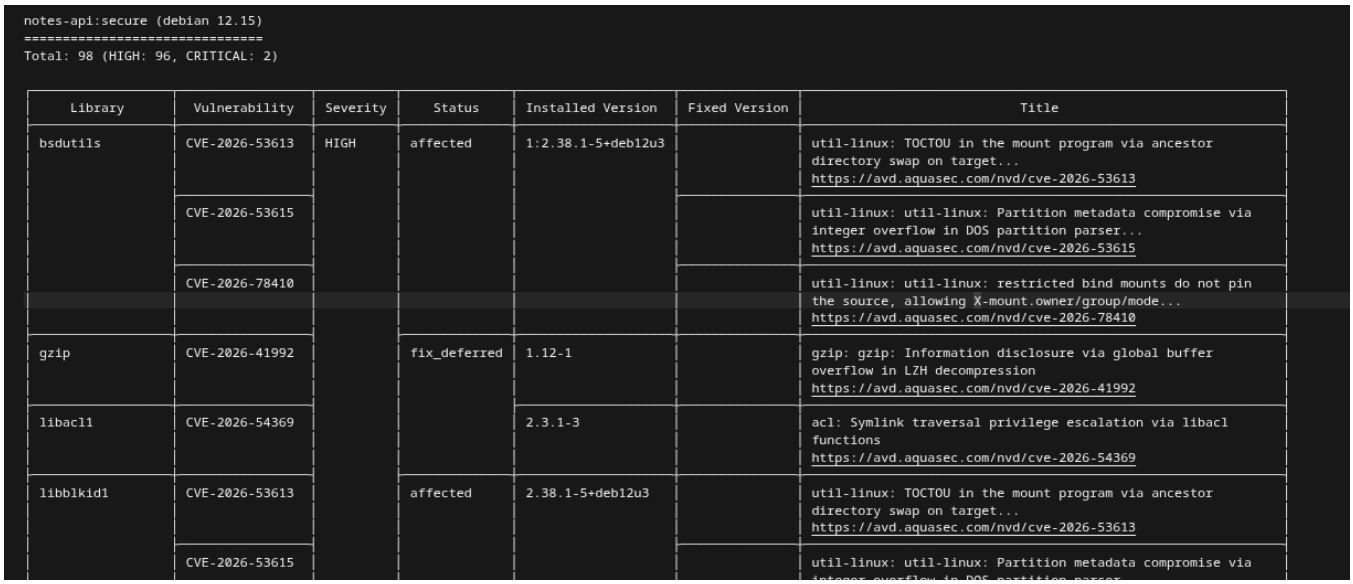
```
(base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/TP3$ docker build -f Dockerfile -t notes-api:secure .
docker inspect notes-api:secure --format '{{.Config.User}} {{.Config.Env}}'
[+] Building 0.5s (11/11) FINISHED                                                
```



Report Summary

Target	Type	Vulnerabilities	Secrets
notes-api:secure (debian 12.15)	debian	98	-
usr/local/lib/python3.12/site-packages/MarkupSafe-3.0.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/blinker-1.9.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/click-8.1.8.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/flask-3.0.3.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/itsdangerous-2.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/jinja2-3.1.6.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/pip-25.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/werkzeug-3.0.6.dist-info/METADATA	python-pkg	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)



notes-api:secure (debian 12.15)
==========
Total: 98 (HIGH: 96, CRITICAL: 2)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
bsdutils	CVE-2026-53613	HIGH	affected	1:2.38.1-5+deb12u3		util-linux: TOCTOU in the mount program via ancestor directory swap on target... https://avd.aquasec.com/nvd/cve-2026-53613
	CVE-2026-53615					util-linux: util-linux: Partition metadata compromise via integer overflow in DOS partition parser... https://avd.aquasec.com/nvd/cve-2026-53615
	CVE-2026-78410					util-linux: util-linux: restricted bind mounts do not pin the source, allowing X-mount.owner/group/mode... https://avd.aquasec.com/nvd/cve-2026-78410
gzip	CVE-2026-41992		fix_deferred	1.12-1		gzip: gzip: Information disclosure via global buffer overflow in LZH decompression https://avd.aquasec.com/nvd/cve-2026-41992
libacl1	CVE-2026-54369			2.3.1-3		acl: Symlink traversal privilege escalation via libacl functions https://avd.aquasec.com/nvd/cve-2026-54369
libblkid1	CVE-2026-53613		affected	2.38.1-5+deb12u3		util-linux: TOCTOU in the mount program via ancestor directory swap on target... https://avd.aquasec.com/nvd/cve-2026-53613
	CVE-2026-53615					util-linux: util-linux: Partition metadata compromise via

	Image faible	**Image durcie**
Base	`python:3.9` (Debian 13.1)	`python:3.12-slim-bookworm` (Debian 12.15)
Utilisateur	root	`appuser` (uid 10001)
Secret dans l'image	`API_KEY` en `ENV`	absent
OS, HIGH + CRITICAL	2774 (2539 + 235)	98 (96 + 2)
Paquets Python, HIGH	3	0

Les 98 vulnérabilités restantes sont des CVE de paquets Debian encore présents dans l'image slim (`bsdutils`, `gzip`, `libc6`). Elles ne se corrigent pas dans le Dockerfile de l'application : elles partent quand la base publie une mise à jour. Le gain mesuré est le passage de 2774 à 98, et la disparition du secret et du root.

---

4. Secret Kubernetes

`app.py` ne contient pas la clé. Il la lit avec `os.environ.get("API_KEY", "")`. La route `/status` ne renvoie que `secret_configured` et `secret_length`.

```python
DB = os.environ.get("DB_PATH", "/tmp/users.db")
API_KEY = os.environ.get("API_KEY", "")
```

```python
@app.route("/status")
def status():
    # Prove the secret is present without printing it.
    return {
        "secret_configured": bool(API_KEY),
        "secret_length": len(API_KEY),
    }
```

Le cluster est un Minikube local. L'image `notes-api:secure` y est chargée avec `minikube image load`, car elle n'est pas sur un registre.

```
(base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/TP3$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Le Secret `notes-api-secret` dans le namespace `notes` porte la clé en `stringData`. Kubernetes la stocke ; elle n'est pas dans le dépôt comme valeur utilisée par l'image, et elle n'est pas dans les layers.

TP3 › k8s › ! secret.yaml

You, 56 minutes ago | author (You)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: notes-api-secret
  namespace: notes
type: Opaque
stringData:
  API_KEY: super-secret-lab-key
```

Le Deployment fixe `runAsNonRoot` et `runAsUser: 10001`, retire toutes les capabilities, et injecte la clé avec `secretKeyRef` sur le Secret précédent.

```yaml
containers:
  - name: notes-api
    image: notes-api:secure
    imagePullPolicy: IfNotPresent
    ports:
      - containerPort: 5000
    env:
      - name: APP_HOST
        value: "0.0.0.0"
      - name: API_KEY
        valueFrom:
          secretKeyRef:
            name: notes-api-secret
            key: API_KEY
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: false
      capabilities:
        drop: ["ALL"]
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
```

Après `kubectl apply`, le pod `notes-api` est `Running` et le Secret est présent dans le namespace.

```
(base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/TP3$ minikube kubectl -- -n notes get pods,secret
NAME                             READY   STATUS    RESTARTS   AGE
pod/notes-api-576f6d8f6b-s7d9h   1/1     Running   0          45s

NAME                         TYPE     DATA   AGE
secret/notes-api-secret      Opaque   1      45s
```

Un `port-forward` vers le Service, puis `curl http://127.0.0.1:5000/status`, renvoie :

```
{"secret_configured": true, "secret_length": 20}
```

La longueur 20 correspond à la clé du Secret. La valeur elle-même n'apparaît pas dans la réponse.

```
(base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs$ curl -s http://127.0.0.1:5000/status
{"secret_configured":true,"secret_length":20}
```

---

5. Falco

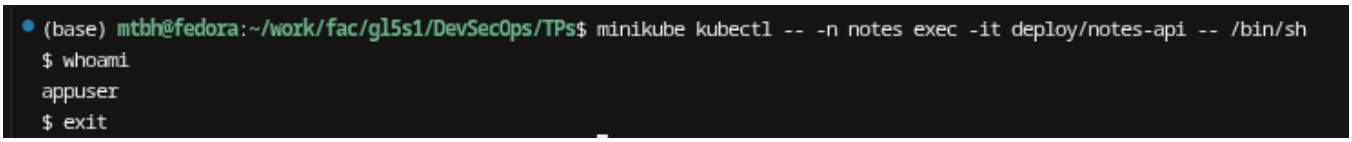
Falco est installé dans le namespace `falco` avec Helm (`driver.kind=modern_ebpf`). Le pod est `Running`.



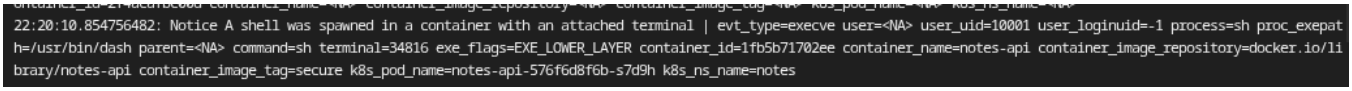
Le comportement simulé est un shell interactif dans le pod de production :

```
minikube kubectl -- -n notes exec -it deploy/notes-api -- /bin/sh
whoami
```

`whoami` affiche `appuser`, ce qui confirme que le shell obtenu n'est pas root.



Falco émet l'alerte au moment où le shell est créé (`execve` de `sh`), pas au moment de `whoami`. La règle observée est **A shell was spawned in a container with an attached terminal**. La ligne indique `user_uid=10001`, `container_name=notes-api`, `container_image_tag=secure`, `k8s_pod_name=notes-api-576f6d8f6b-s7d9h` et `k8s_ns_name=notes`.



Minikube avec le driver Docker partage le noyau de la machine. La sonde eBPF voit donc aussi des conteneurs Docker hors cluster. Leurs événements ont `k8s_pod_name=<NA>`. L'alerte retenue pour le TP est celle dont le namespace est `notes`.

---

6. Bilan

- L'image passe de `python:3.9` en root, avec le secret dans `ENV`, à `python:3.12-slim` exécutée par `appuser`, sans secret dans les layers. Trivy mesure le résultat : 2774 vulnérabilités HIGH/CRITICAL sur l'OS, puis 98.
- La clé est fournie au pod par un Secret Kubernetes (`secretKeyRef`). `/status` prouve qu'elle est disponible sans l'afficher.
- Falco, déployé sur le cluster, signale le shell ouvert dans `notes-api` par la règle *A shell was spawned in a container with an attached terminal*.